

設計ドキュメント

1. サービス概要

目的・課題解決

- RSS フィードから最新の技術情報を取得し、自動的に要約・分類する。
- 生成 AI を活用して、英語・日本語の技術情報をタグ付けし、ユーザーが手軽に情報を得られるようにする。
- 要約した情報をユーザーに通知する。

想定ユーザー

- エンジニア
- 初期段階では開発者自身が利用

主要機能

- RSS から最新の技術情報を取得
- 生成 AI によるタグ付けと要約
- ユーザーへの通知

2. アーキテクチャ

記事「[私のよく使うソフトウェアアーキテクチャの雛型](#)」を参考に、以下の4層構造を採用。

- **プレゼンテーション層**: ソフトウェアの入出力を担当
- **アプリケーション層**: ソフトウェアのユースケースを担当
- **ドメイン層**: ビジネスのルールや制約、プロセスを担当
- **インフラストラクチャー層**: 技術的関心ごとの全般を担当

技術スタック

- **フロントエンド**: TypeScript, React, Next.js
- **バックエンド**: Golang
- **生成 AI**: DeepSeek API
- **インフラ**: AWS CDK v2 (TypeScript)

クラウドサービス

- AWS (無料枠を最大限活用)

データベース

- NoSQL (DynamoDB)

デプロイ

- サーバーレスアーキテクチャを優先
- AWS CDK v2 を使用し、インフラをコード管理 (IaC)

- フロントエンドは Next.js の静的サイトを S3 + CloudFront でホスティング

ローカル環境の構築

ローカル環境の代替ツール

AWS サービス	ローカル開発ツール
AWS Lambda	RIE (AWS Lambda Runtime Interface Emulator)
DynamoDB	DynamoDB Local
S3	MinIO
EventBridge	手動で Lambda を実行
API Gateway	不要

Docker を活用したローカル開発環境

```
repo-root/
├── frontend/      # Next.js (TypeScript)
├── backend/       # Golang (Lambda Functions)
├── infra/         # AWS CDK v2 (TypeScript)
│   ├── bin/
│   ├── lib/
│   ├── stacks/
│   └── cdk.json
├── local/         # ローカル開発環境用
│   ├── docker-compose.yml
│   ├── scripts/  # MinIO, RIE, DynamoDB Local の起動スクリプトなど
│   └── .github/   # GitHub Actions
│       ├── workflows/
│       │   ├── deploy.yml # CI/CD Pipeline
│       │   └── test.yml   # Lint & Unit Test
```

local/docker-compose.yml

```
version: "3.8"

services:
  dynamodb:
    image: amazon/dynamodb-local
    container_name: dynamodb
    ports:
      - "8000:8000"
    command: "-jar DynamoDBLocal.jar -sharedDb"

  minio:
    image: minio/minio
```

```

    container_name: minio
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    command: server /data --console-address ":9001"
    volumes:
      - minio_data:/data

  lambda:
    build:
      context: ../backend
      dockerfile: Dockerfile
    container_name: lambda
    env_file:
      - .env
    volumes:
      - ../backend:/app
    command: ["/app/entry.sh"]

volumes:
  minio_data:

```

CI/CD とローカル環境の統合

GitHub Actions ワークフロー例 (test.yml)

```

name: Run Local Environment Tests
on:
  push:
    branches:
      - main
jobs:
  test-local:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v2
      - name: Start Local Services
        run: docker-compose -f local/docker-compose.yml up -d
      - name: Run Tests
        run: cd backend && go test ./...

```

まとめ

- 生成AIを DeepSeek API に変更し、コストを抑えつつ高品質な要約を実現
- Next.js を S3 + CloudFront でホスティングし、静的サイトとして提供

- RIE で AWS Lambda 環境をローカルで再現
- MinIO を S3 の代替として利用
- DynamoDB Local でデータ管理
- CI/CD にローカル環境のテストを組み込み、安定したデプロイを実現

5. データベース設計

✅ **articles** テーブル (要約記事を保存)

Partition Key (id - UUIDv7)	Url	Title	Summary	Tags (with Score)	Created At
uuid-xxx	https://example.com/article1	"AI の未 来"	"要約結 果..."	[{ name: "AI", score: 0.92 }]	2025- 03-15

✅ **users** テーブル (ユーザー管理)

Partition Key (user_id)	Plan	Requests Limit	Used Requests	Last Reset
user123	free	5	3	2025-03-15
user456	premium	100	10	2025-03-15

✅ **tags** テーブル (タグ情報)

Partition Key (tag_name)	Created At
ai	2025-03-10
cloud	2025-03-10

※ 表記ゆれ防止のため、**tag_name** はすべて小文字化して保存

✅ **article_tags** テーブル (タグと記事の関連 + スコア)

Partition Key (tag_name)	Sort Key (article_id)	Score
ai	article_abc123	0.92
cloud	article_abc123	0.80
security	article_def456	0.75

タグごとの関連スコアを記録することで、精度の高い検索・推薦・分析が可能になります。